

Project - High Level Design

on

Entertainment Planning Assistant Agent

Course Name: AGENTIC AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	MRUNALI KAMERIKAR	EN22CS301618
2	MOHD QUASIM	EN22CS301603
3	MUSKAN ASIJA	EN22CS301623
4	MAITRY BANDUKE	EN22CS301571
5	NIKHIL KUMAR SINGH	EN22CS301648

Group Name: Group 03D6

Project Number: AAI-14

Industry Mentor Name:

University Mentor Name: NISHANT SHRIVASTAV

Academic Year: 2026

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

The **Entertainment Planning Assistant Agent** is an AI-powered Planner Agent designed to autonomously orchestrate complex entertainment-related tasks through a structured multi-step reasoning loop.

The system accepts a high-level goal such as:

- "Movie Reviews"
- "Create weekend watchlist plan"
- "Generate OTT recommendation schedule"

The agent then:

- Decomposes the goal into actionable subtasks
- Identifies task dependencies
- Validates resources using mock tools
- Optimizes task sequencing
- Produces a detailed execution schedule

The system is implemented using:

- **Python**
- LangChain / CrewAI
- LLM API
- Mock Tool Interfaces

The primary objective is to demonstrate intelligent task planning, dependency handling, and reasoning in a dynamic environment.

1.1 Scope of the Document

This document covers the **High-Level Design (HLD)** of the Entertainment Planning Assistant Agent.

It includes:

- System architecture
- Application design
- Process and information flow
- Component-level design
- API catalog
- Data design
- Interface specifications
- State/session management
- Caching strategy
- Non-functional requirements

Out of Scope:

- UI/Frontend design
- Cloud deployment infrastructure
- Production monitoring setup
- CI/CD pipeline implementation
- Detailed Low-Level Design (LLD)

1.2 Intended Audience

This document is intended for:

- Software developers implementing the Planner Agent
- AI/ML engineers integrating LLM APIs
- System architects reviewing solution design
- Academic evaluators (for project submission)
- Technical stakeholders assessing system feasibility

It assumes basic knowledge of:

- Python programming
- Agent frameworks (LangChain/CrewAI)
- REST APIs
- LLM-based systems

1.3 System Overview

The Entertainment Planning Assistant Agent is a reasoning-driven system that operates using a **Plan** → **Validate** → **Optimize** → **Execute** approach.

Core Functional Flow:

1. User provides a high-level entertainment goal.
2. The Planner Agent interprets intent using LLM.
3. The system decomposes the goal into structured subtasks.
4. Dependencies are mapped using a DAG (Directed Acyclic Graph).
5. Mock tools validate resource availability.
6. The task sequence is optimized.
7. A detailed execution schedule is generated.



High-Level Architecture Overview

2. System Design

2.1 Application Design

2.1.1 Architectural Style

The Entertainment Planning Assistant Agent follows a **Modular Layered Architecture with Agent-Oriented Design**.

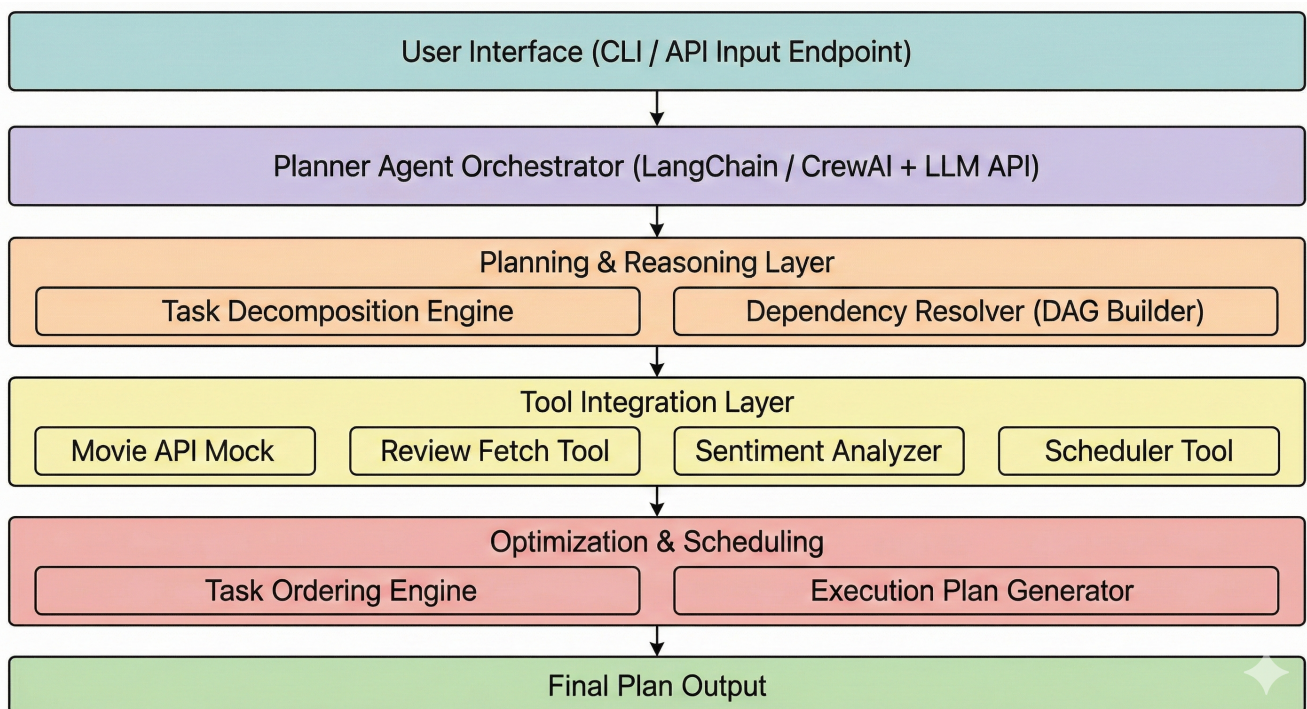
The architecture is divided into logical layers:

1. **Presentation Layer** (Input/Output handling)
2. **Agent Orchestration Layer**
3. **Planning & Reasoning Layer**
4. **Tool Integration Layer**
5. **Scheduling & Optimization Layer**
6. **Data Management Layer**

This separation ensures:

- High maintainability
- Easy module replacement
- Clear separation of concerns
- Extensibility for future enhancements

2.1.2 High-Level Application Architecture



2.1.3 Core Modules

1. Planner Agent Orchestrator

- Controls reasoning loop
- Manages memory/state
- Calls tools dynamically
- Handles error recovery

- Python
- LangChain / CrewAI
- LLM API

2. Task Decomposition Module

- Converts high-level goal into structured subtasks
- Produces JSON-based task objects
- Assigns task metadata (priority, estimated time, dependencies)

Example Output:

```
{  
  "task_id": "T1",  
  "description": "Fetch trending action movies",  
  "depends_on": [],  
  "estimated_time": 5  
}
```

3. Dependency Resolver

- Builds Directed Acyclic Graph (DAG)
- Detects cycles
- Applies topological sorting
- Ensures correct execution order

4. Tool Execution Layer

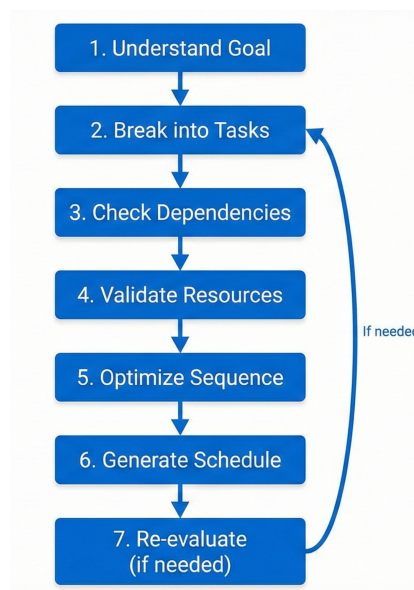
- Simulates external systems
- Validates data/resource availability
- Returns structured responses
- Implements retry mechanism

5. Schedule Optimizer

- Evaluates task sequence
- Optimizes parallel execution
- Generates time-based schedule
- Minimizes idle time

2.1.4 Multi-Step Reasoning Loop Design

The Planner Agent operates using an iterative reasoning loop:



Loop Termination Conditions:

- Goal achieved
- Maximum iteration reached
- Critical failure encountered

2.1.5 Deployment Model (Logical)

For academic/project purposes:

- Single Python application
- Runs locally
- LLM API accessed via REST
- Mock tools implemented as Python classes

Future-ready for:

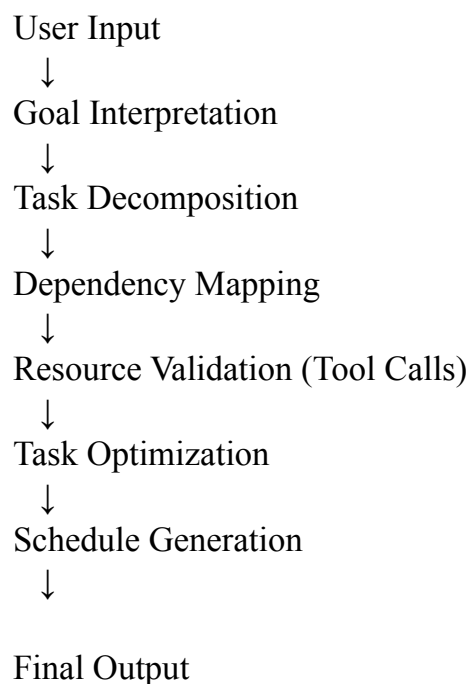
- Microservices architecture
- Cloud deployment
- Multi-agent scaling

2.2 Process Flow

This section describes the **end-to-end operational flow** of the Entertainment Planning Assistant Agent from user input to final execution schedule generation.

2.2.1 High-Level Process Flow

The system follows a structured multi-step reasoning process:



2.2.2 Detailed Step-by-Step Process

Step 1: User Goal Submission

The user provides a high-level entertainment objective.

Example:

“Create a movie review plan for latest action movies.”

Input can be received via:

- CLI
- REST API
- Agent interface

Step 2: Goal Interpretation

- The LLM interprets user intent.
- Extracts domain and constraints.
- Identifies expected deliverables.

Example Extracted Intent:

- Domain: Movies
- Category: Action
- Output: Review Plan

Step 3: Task Decomposition

The system breaks the goal into structured subtasks.

Example:

Task ID	Task Description	Dependency
T1	Fetch trending action movies	None
T2	Retrieve reviews	T1
T3	Perform sentiment analysis	T2
T4	Generate summary report	T3

Tasks are generated in structured JSON format.

Step 4: Dependency Mapping

- Build a Directed Acyclic Graph (DAG).
- Ensure no circular dependency.
- Determine executable order.

Example:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$

If cycle detected:

- System triggers re-planning.

Step 5: Resource Validation

Each task is validated via mock tools.

Example checks:

- Is movie data available?
- Are reviews accessible?

- Retry logic executed
- If repeated failure → Re-plan

Step 6: Task Optimization

The system:

- Applies topological sorting.
- Identifies parallelizable tasks.
- Estimates total execution time.
- Minimizes idle waiting.

Optimization Goals:

- Reduce total time
- Avoid resource conflict
- Improve execution efficiency

Step 7: Execution Schedule Generation

The system produces a final structured plan.

Example Output:

Execution Plan:

Step 1: Fetch trending action movies (5 min)

Step 2: Retrieve reviews (10 min)

Step 3: Perform sentiment analysis (3 min)

Step 4: Generate summary report (2 min)

Total Estimated Time: 20 minutes

2.2.3 Exception Handling Flow

Scenario 1: Tool Failure

- Retry up to N attempts
- If failure persists → Trigger re-planning

Scenario 2: Dependency Error

- Detect cycle in DAG
- Re-generate task decomposition

Scenario 3: LLM Output Error

- Validate JSON schema
- Re-prompt with structured constraints

2.2.4 Multi-Step Reasoning Loop (Internal Process)

Internally, the agent runs:

While Goal_Not_Achieved:

Think (LLM reasoning)
Plan (Task decomposition)
Validate (Tool calls)
Observe (Results)
Refine (Adjust plan)

This enables:

- Adaptive planning
- Dynamic error recovery
- Intelligent task sequencing

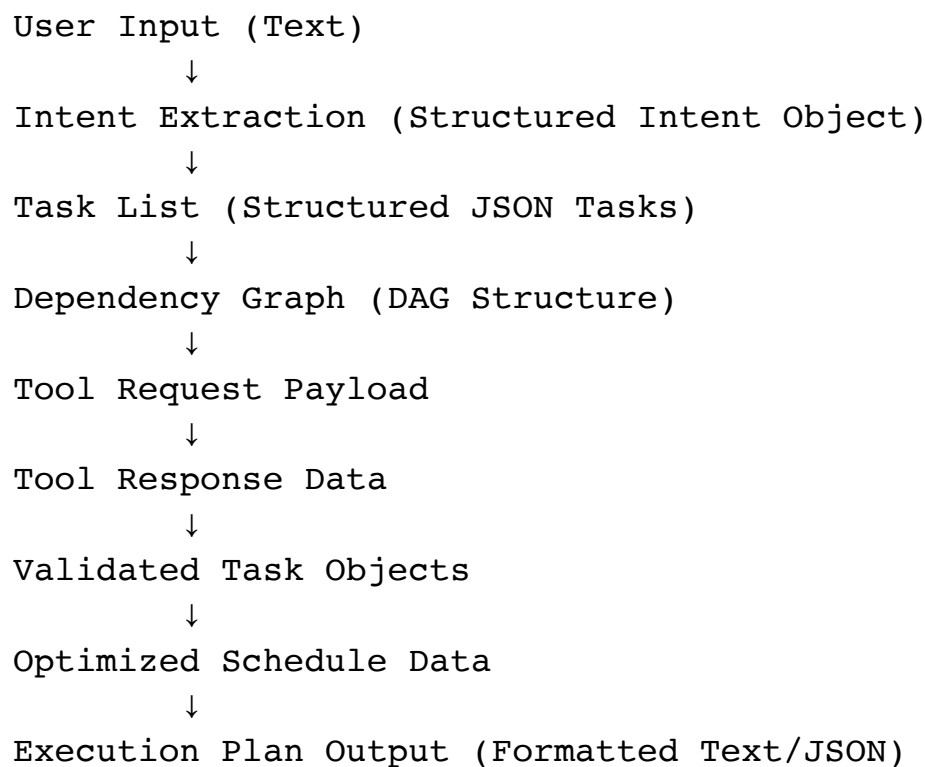
2.3 Information Flow

This section explains how **data and information move across system components** within the Entertainment Planning Assistant Agent.

Unlike Process Flow (which explains steps), Information Flow focuses on:

- What data is generated
- How it is transformed
- Where it is stored
- How it moves between modules

2.3.1 High-Level Information Flow



2.3.2 Information Transformation Stages

1. User Input → Intent Object

Raw Input:

"Generate movie review report for latest action movies"
Transformed into:

```
{
  "domain": "movies",
  "genre": "action",
  "output_type": "review_report"
}
```

The LLM structures unstructured text into machine-readable data.

2. Intent Object → Task Objects

Structured tasks are generated:

```
[
  {
    "task_id": "T1",
    "description": "Fetch trending action movies",
    "depends_on": [],
    "status": "pending"
  },
  {
    "task_id": "T2",
    "description": "Fetch reviews",
    "depends_on": ["T1"],
    "status": "pending"
  }
]
```

Each task contains:

- Unique ID
- Description
- Dependencies
- Status
- Estimated time

3. Task Objects → Dependency Graph

Tasks are converted into a DAG structure:

Nodes: T1, T2, T3, T4

Edges: T1 → T2 → T3 → T4

Stored internally as:

- Adjacency list OR
- Graph object

This enables:

- Cycle detection
- Execution ordering
- Dependency validation

4. Task Objects → Tool Request Payload

Before execution, tasks generate tool calls.

Example Tool Request:

```
{
  "tool_name": "movie_api",
  "parameters": {
    "genre": "action",
    "limit": 10
  }
}
```

5. Tool Response → Validated Task

Tool Response:

```
{
  "status": "success",
  "data": [...movie_list...]
```

```
}
```

Task state updated to:

```
{  
  "task_id": "T1",  
  "status": "completed"  
}
```

6. Validated Tasks → Optimized Schedule

Scheduler receives:

- List of validated tasks
- Dependency graph
- Estimated execution times

Produces:

```
{  
  "execution_order": ["T1", "T2", "T3", "T4"],  
  "total_estimated_time": 20  
}
```

7. Final Output Generation

Converted to user-readable format:

Execution Plan:

1. Fetch trending action movies (5 min)
2. Fetch reviews (10 min)
3. Perform sentiment analysis (3 min)
4. Generate summary report (2 min)

2.4 Components Design

This section describes the internal components of the **Entertainment Planning Assistant Agent**, their responsibilities, internal structure, and interactions.

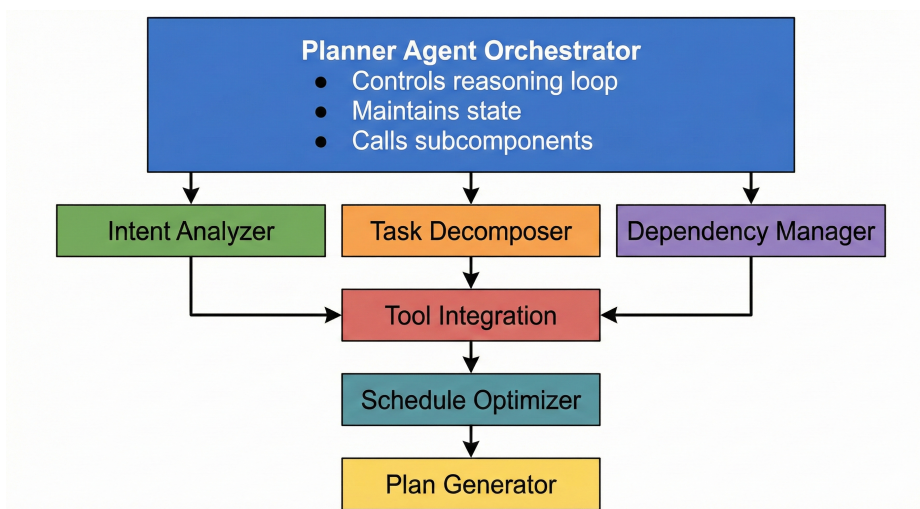
The system is modular and component-driven to ensure scalability and maintainability.

2.4.1 Component Overview

The major components are:

1. Planner Agent Orchestrator
2. Intent Analyzer
3. Task Decomposition Engine
4. Dependency Manager
5. Tool Integration Layer
6. Schedule Optimizer
7. Execution Plan Generator
8. Logging & Monitoring Module

2.4.2 Component-Level Architecture



2.4.3 Detailed Component Description

1. Planner Agent Orchestrator

Responsibility:

- Acts as central controller
- Manages reasoning loop
- Coordinates all modules
- Handles error recovery

Internal Functions:

- `receive_goal()`
- `invoke_decomposer()`
- `validate_tasks()`
- `optimize_schedule()`
- `generate_output()`

Design Choice:

Implemented using:

- LangChain / CrewAI Agent
- LLM with tool-calling capability

2. Intent Analyzer

Responsibility:

- Converts raw user text into structured intent object
- Extracts domain, constraints, output type

Input:

User text

Output:

Structured JSON intent

Example:

```
{  
  "domain": "movies",  
  "genre": "action",  
  "objective": "review_report"  
}
```

3. Task Decomposition Engine

Responsibility:

- Break high-level goal into subtasks
- Assign task IDs
- Define dependencies
- Estimate execution time

Output Structure:

```
{  
  "task_id": "T1",  
  "description": "Fetch trending movies",  
  "depends_on": [],  
  "estimated_time": 5,  
  "status": "pending"  
}
```

Key Features:

- Uses structured prompting
- JSON schema validation
- Retry on malformed output

4. Dependency Manager

Responsibility:

- Build Directed Acyclic Graph (DAG)
- Detect cycles
- Perform topological sorting
- Provide execution order

Internal Representation:

- Adjacency list
- Node-based graph object

Key Algorithms:

- Cycle detection (DFS)
- Topological sort (Kahn's Algorithm)

5. Tool Integration Layer

Responsibility:

- Simulate external APIs
- Validate task feasibility
- Return structured responses
- Implement retry logic

Example Tools:

- Movie Data Tool
- Review Fetch Tool
- Sentiment Analysis Tool
- Scheduler Tool

Tool Response Format:

```
{  
  "status": "success",  
  "data": {...}  
}
```

6. Schedule Optimizer**Responsibility:**

- Optimize task order
- Detect parallel tasks
- Estimate total execution time
- Avoid resource conflicts

Optimization Logic:

- Topological sorting
- Parallel grouping
- Greedy time minimization

7. Execution Plan Generator**Responsibility:**

- Convert structured schedule into user-readable plan

- Format output (text or JSON)

Example Output:

```
Step 1: Fetch trending movies (5 min)
Step 2: Fetch reviews (10 min)
Step 3: Analyze sentiment (3 min)
Step 4: Generate summary (2 min)
```

8. Logging & Monitoring Module

Responsibility:

- Log reasoning steps
- Track tool calls
- Capture errors
- Maintain execution trace

This ensures:

- Debugging capability
- Academic evaluation clarity
- Observability

2.5 Key Design Considerations

This section outlines the major architectural and technical decisions taken while designing the **Entertainment Planning Assistant Agent**.

These considerations ensure scalability, robustness, maintainability, and intelligent reasoning capability.

2.5.1 Modularity and Separation of Concerns

The system is divided into independent modules:

- Intent Analyzer

- Task Decomposer
- Dependency Manager
- Tool Integration Layer
- Schedule Optimizer
- Output Generator

Why?

- Easier debugging
- Independent testing
- Replaceable components
- Future extensibility

Example:

Mock tools can later be replaced with real APIs without affecting the planner logic.

2.5.2 Structured LLM Output

LLM responses are enforced in structured JSON format.

Reason:

- Prevent hallucinated unstructured text
- Enable machine validation
- Simplify dependency graph construction

Validation techniques:

- JSON schema validation
- Retry with corrective prompts

2.5.3 Dependency-Driven Planning (DAG-Based Design)

Tasks are modeled as a Directed Acyclic Graph (DAG).



- Prevent circular dependencies
- Enable correct execution order
- Allow parallel execution
- Detect logical inconsistencies

Algorithms Used:

- Cycle detection (DFS)
- Topological sorting

2.6 API Catalogue

This section describes the internal and external APIs used in the Entertainment Planning Assistant Agent system.

2.6.1 Goal Submission API

Endpoint: /submit-goal

Method: POST

Description: Accepts a high-level entertainment planning goal from the user.

Request:

```
{  
  "goal": "Movie Reviews for latest releases"  
}
```

Response:

```
{  
  "task_id": "12345",  
  "status": "Planning Initiated"  
}
```

Purpose:

- Triggers Planner Agent
- Initiates reasoning loop



Endpoint: /task-status/{task_id}

Method: GET

Description: Retrieves the current execution state of the planning task.

Response:

```
{
  "task_id": "12345",
  "status": "Execution Scheduled",
  "progress": "70%"
}
```

2.6.3 Resource Validation API (Mock Tool)

Endpoint: /validate-resource

Method: POST

Description: Simulates validation of external entertainment resources (e.g., availability of movie database, show timings, streaming availability).

Request:

```
{
  "resource_type": "Movie Database",
  "resource_name": "IMDb"
}
```

Response:

```
{
  "available": true,
  "latency": "120ms"
}
```

2.6.4 LLM Interaction API

External Integration: LLM Provider (e.g., OpenAI API)

Purpose:

- Task decomposition
- Dependency resolution
- Schedule generation
- Optimization reasoning

Invocation Example (Conceptual):

```
llm.invoke(prompt)
```

3. Data Design

The data design focuses on storing planning goals, decomposed tasks, dependencies, execution schedules, and system metadata.

3.1 Data Model

The system follows a structured object-based model.

3.1.1 Entities

1. Goal

- goal_id (UUID)
- goal_description (Text)
- created_at (Timestamp)
- status (Enum: Initiated, Planning, Scheduled, Completed)

2. Task

- task_id (UUID)
- goal_id (Foreign Key)
- task_description (Text)

- priority (Integer)
- dependency_ids (List)
- status (Pending, Validated, Scheduled, Completed)

3. Resource

- resource_id
- resource_name
- resource_type
- availability_status

4. Execution Schedule

- schedule_id
- goal_id
- ordered_tasks (List)
- estimated_completion_time

3.2 Data Access Mechanism

The system uses:

- In-memory storage (for prototype implementation)
- Python dictionaries / JSON structures
- Optional SQLite for persistent storage

Access Layer Responsibilities:

- CRUD operations on goals and tasks
- Maintain dependency DAG
- Update task status during execution
- Retrieve schedules efficiently

LangChain/CrewAI memory modules may also maintain intermediate reasoning state.



3.3 Data Retention Policies

For Prototype Version:

- Data stored temporarily in memory
- Cleared after session termination

For Production Version:

- Retain goal and task history for 30–90 days
- Logs stored for auditing and debugging
- Sensitive data anonymized

Backup Strategy:

- Periodic snapshot of schedules
- JSON export of execution plans

3.4 Data Migration

Future migration plan:

- Move from in-memory storage to relational database (PostgreSQL)
- Schema versioning for data model evolution
- Use migration tools like Alembic (if SQL-based)
- Backward compatibility maintained via version-controlled schema

4. Interfaces

4.1 User Interface

- CLI-based interaction (initial version)
- Optional REST API endpoint
- Future: Web-based dashboard

User Inputs:

- High-level entertainment goal
- Constraints (optional time, platform preference)

Outputs:

- Task breakdown
- Dependency graph
- Execution timeline

4.2 External Interfaces

1. LLM API
2. Mock Resource Validation Tools
3. Optional future integrations:
 - Movie databases
 - Streaming platforms APIs

5. State and Session Management

The Planner Agent operates in a session-based context.

Session Data Includes:

- Current goal
- Intermediate reasoning steps
- Generated tasks
- Dependency graph
- Execution progress

State Management Strategy:

- Stored in memory during session

- LangChain conversation memory maintains reasoning history
- Unique task_id binds session state

Failure Handling:

- On failure, system retries reasoning loop
- Partial plan recovery supported

6. Caching

Caching improves performance and reduces LLM API cost.

Cacheable Components:

- Previously decomposed similar goals
- Resource validation results
- LLM responses for repeated prompts

Caching Strategy:

- In-memory dictionary cache
- Optional Redis (future enhancement)
- Cache invalidation after configurable TTL

Benefits:

- Reduced API latency
- Cost optimization
- Faster repeated planning tasks

7. Non-Functional Requirements

7.1 Security Aspects

1. API authentication (Token-based authentication)
2. Secure storage of LLM API keys
3. Rate limiting to prevent abuse
4. Input validation to prevent prompt injection
5. Logging with masked sensitive information

Threat Mitigation:

- Sanitize user inputs before sending to LLM
- Restrict tool invocation permissions
- Controlled execution environment for Python tools

7.2 Performance Aspects

Target Performance Goals:

- Goal decomposition < 3 seconds
- Resource validation < 500 ms (mock)
- Schedule generation < 5 seconds

Scalability:

- Horizontal scaling via microservice architecture
- Asynchronous task execution
- Load balancing for API layer

Optimization Techniques:

- Parallel resource validation
- Efficient DAG construction
- LLM prompt optimization

8. References

1. LangChain Documentation – <https://python.langchain.com>
2. CrewAI Documentation – <https://docs.crewai.com>
3. OpenAI API Documentation – <https://platform.openai.com/docs>
4. REST API Design Best Practices – Richardson Maturity Model